College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID608001: Intermediate Application Development Concepts
Level 6, Credits 15

Project

Assessment Overview

In this **individual** assessment, you will design and develop **two** web applications.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Apply design patterns and programming principles using software development best practices.
2. Design and implement full-stack applications using industry relevant programming languages.

Assessments

Assessment	Weighting	Due Date	Learning Outcomes
Practical	20%	Sunday 9th November at 11.59 PM	1
Project	80%	Sunday 9th November at 11.59 PM	1 and 2

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Sunday 9th November** at **11.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID608001: Intermediate Application Development Concepts**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcomes for this assessment.

Learning to use AI tools is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Submission

You **must** submit all files via **GitHub**. Create a repository and add the course lecturer as a collaborator. The latest files will be used to mark against the marking rubric. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **12.00 AM**.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID608001: Intermediate Application Development Concepts**.

Functionality - Learning Outcomes 1, 2 (25%)

- **HackerNews API** application with the following functionality:
  - Data is fetch from the HackerNews API.
  - Display a spinner while the data is being fetched from the API.
  - Responsive UI design using **Tailwind CSS**.
  - **Story Features:**
    - Display the 50 ask, best, job, new, show and top stories.
    - For each story, display the title, by, text, time and a link to the story using the id. Here is an example link - https://news.ycombinator.com/item?id=ID, where ID is the story's id.
    - Truncate the text to 100 characters and add a link that expands the text when clicked.
  - **Leader Features:**
    - Display 100 leaders. Here is a link to the leaders - https://news.ycombinator.com/leaders.
    - For each leader, display their id, karma, created, about and a link to their profile using the id. Here is an example link - https://news.ycombinator.com/user?id=ID, where ID is the user's id.
  - **Favourites Features:**
    - Display a heart icon next to the story or leader. Toggle the heart icon when clicked to add or remove the story or leader from favourites.
    - Display a list of favourite stories and leaders.
    - If no favourites are found, display a message indicating that no favourites were found.
  - **Search Features:**
    - Search for stories by title or type.
    - Search for leaders by id.
    - The search should be case-insensitive and return results that contain the search term anywhere in the title, type or id.
    - Display the search results in a list.
    - If no results are found, display a message indicating that no results were found.
  - **Sort Features:**
    - Sort stories by time, score or type.
    - Sort leaders by karma or created.
    - The default sort order for stories is time and for leaders is karma.
  - Display the time and created in a human-readable format.
  - Select any five **Medium Features** from the list below:
    - Hover effects on stories and leaders.
    - Toggle between colour themes.
    - Display the domain name from the story's URL. For example, if the story's URL is `https://example.com/story`, display `example.com`.
    - Display score-range badges with colours. For example, 0-100 is red, 101-500 is orange, 501-1000 is yellow, 1001-5000 is green and 5001+ is blue.
    - Display type badges with colours. For example, ask is blue, best is green, job is yellow, new is purple, show is orange and top is red.
    - Select and compare two leaders. Display their id, karma, created, about and a link to their profile using the id.
    - Display karma-range badges with colours for leaders. For example, 0-100 is red, 101-500 is orange, 501-1000 is yellow, 1001-5000 is green and 5001+ is blue.
    - For stories, toggle between list and grid view. The list view should display the title, by, text, time and a link to the story using the id. The grid view should display the title, by and a link to the story using the id.
  - Select any two **Hard Features** from the list below:

    - Display the stories and leaders in pages of 10 with ”Previous” and ”Next” navigation buttons.
    - Display stories and leaders search history based on the search functionality.
    - Display a graph of the karma for each leader. Display their id on the x-axis and karma on the y-axis.
    - Display a list of recommended stories based on the user’s favourites. The recommendations should be based on the user’s favourites and the stories’ type.

- **OpenTDB API** application with the following functionality:
    - Data is fetch from the https://opentdb.com/api_config.php.
    - Display a spinner while the data is being fetched from the API.
    - Responsive UI design using **Shadcn UI**.
    - Create a new quiz by selecting the start date, end date, number of questions, category, difficulty and type.
    - Display a list of past, current and future quizzes.
    - Past and future quizzes are not playable.
    - Play a quiz by selecting a current quiz from the list. The quiz should display the start date, end date, category, difficulty, type, questions, options and a submit button.
    - Shuffle the questions and options for each quiz.
    - Display the quiz results after submitting the quiz. Prompt the user to enter their name before displaying the results. The results should display the user’s name, start date, end date, category, difficulty, type, questions, options, correct answer and score.
    - Store the quiz results appropriately.
    - Display the average score for each quiz.
    - Display the total number of quizzes played.
    - For each quiz, display the scores in a list.
    - Select any three **Medium Features** from the list below:
        - Display a progress bar and ”Question X of Y” text while playing the quiz.
        - Display category badges with colours. For example, general knowledge is blue, art is green, history is yellow, geography is purple, etc.
        - Allow a user to retake a completed quiz.
        - For each quiz, display a countdown timer per question. The timer should start at ten minutes. If the timer reaches 0, the quiz should automatically submit.
        - Select and compare two users. Display their name, number of quizzes played and average score.
- Deploy both applications to **Vercel**.

Code Quality and Best Practices - Learning Outcomes 1, 2 (40%)

- Write clean, readable, and maintainable code.
- Use modular, reusable components and avoid code duplication.
- Keep code simple and easy to understand.
- Optimise performance by managing resources efficiently.

Version Control - Learning Outcome 1 (5%)

- Regular commits are made throughout development, demonstrating meaningful progress.
- Commit messages are clear and descriptive.
- The repository shows a clean and logical commit history that reflects the development process of the applications.

Reflection - Learning Outcomes 1, 2 (5%)

- Write a short reflection (300-500 words) on your development process and learning throughout the project, including:
  - Challenges you faced and how you overcame them.
  - What you would do differently in future projects.
  - New skills, tools or practices you learned during the project.

Presentation - Learning Outcomes 1, 2 (5%)

- Record a 5-10 minute screen recording of your **HackerNews API** and **OpenTDB API** applications.
- Demonstrate the applications' features.
- Make sure the video is clear and easy to follow.
- Submit a link to the presentation in your repository's **README.md** file.